

Generics3: Generics und Polymorphie

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation: Wie verhalten sich generische Typen zueinander

```
interface Animal {  
    default void eat() {}  
}  
  
record Dog() implements Animal {}  
record Cat() implements Animal {}
```

Da `Cat <: Animal` gilt: Ist dann auch `List<Cat> <: List<Animal>`?

```
List<Animal> animals = new ArrayList<Cat>(); // ???
```

Invarianz generischer Klassen in Java

```
List<Cat> cats = new ArrayList<>();  
List<Animal> animals = cats; // hypothetisch erlaubt  
  
animals.add(new Dog());      // erlaubt, weil animals: List<Animal>  
Cat c = cats.getFirst();     // aber jetzt steckt ein Dog in cats!
```

Invarianz generischer Klassen in Java

```
List<Cat> cats = new ArrayList<>();  
List<Animal> animals = cats; // hypothetisch erlaubt  
  
animals.add(new Dog());      // erlaubt, weil animals: List<Animal>  
Cat c = cats.getFirst();     // aber jetzt steckt ein Dog in cats!
```

Widerspruch zur Typsicherheit

Important

Java-Generics sind **invariant**

Wenn `A <: B` (`A` Untertyp von `B`), dann folgt **nicht** `List<A> <: List<B>`.

Kovarianz mit `? extends` (Producer: nur lesen)

```
static void feedAll(List<? extends Animal> animals) {  
    for (Animal a : animals) { a.eat(); }  
  
    // animals.add(new Cat()); // Compiler-Fehler  
}  
  
List<Cat> cats = ... ; feedAll(cats);
```

Kovarianz mit `? extends` (Producer: nur lesen)

```
static void feedAll(List<? extends Animal> animals) {  
    for (Animal a : animals) { a.eat(); }  
  
    // animals.add(new Cat()); // Compiler-Fehler  
}  
  
List<Cat> cats = ... ; feedAll(cats);
```

Important

Kovarianz durch `? extends T`:

- “Ich akzeptiere Listen von Untertypen von `T`”
- Nur sicher als **Producer** von `T` (lesen erlaubt)
- `List<Cat>` ist **kein** Untertyp von `List<Animal>`
- Aber: `List<Cat> <: List<? extends Animal>`

Kontravarianz mit `? super` (Consumer: nur schreiben)

```
static void addCats(List<? super Cat> cats) {  
    cats.add(new Cat());           // erlaubt  
    // Cat c = cats.getFirst(); // Compiler-Fehler: Rückgabetyt ist Object  
}  
  
List<Cat> cats = ... ; addCats(cats);
```

Kontravarianz mit `? super` (Consumer: nur schreiben)

```
static void addCats(List<? super Cat> cats) {  
    cats.add(new Cat());           // erlaubt  
    // Cat c = cats.getFirst(); // Compiler-Fehler: Rückgabetyp ist Object  
}  
  
List<Cat> cats = ... ; addCats(cats);
```

Important

Kontravarianz durch `? super T`:

- “Ich akzeptiere Listen von Supertypen von `T`”
- Nur sicher als **Consumer** von `T` (schreiben/hinzufügen erlaubt)
- `? super Cat` bedeutet: Typ-Argument ist `Cat`, `Animal` oder `Object`
- `List<Animal> <: List<? super Cat>`

PECS: Producer Extends, Consumer Super

- **Producer** (liefert Elemente, **kovariant**): `List<? extends Animal>`
- **Consumer** (nimmt Elemente auf, **kontravariant**): `List<? super Cat>`
- Ohne Wildcard `List<Cat>`: Typ ist **invariant**; Lesen und Schreiben für genau diesen Typ

Bezug zu funktionalen Interfaces

- `Function<? super T, ? extends R>`
 - Abbildung von `T` nach `R`: `R apply(T)`
 - Input (konsumiert `T`): `? super T`
 - Output (produziert `R`): `? extends R`
- `Comparator<? super T>`
 - `int compare(T, T)`
 - Ein `Comparator` vergleicht `T`-Objekte, "konsumiert" also `T`: `? super T`

Typ-Löschung (*Type Erasure*)

```
class Studi<T> {  
    T myst(T m, T n) { return n; }  
  
    public static void main(String[] args) {  
        Studi<Integer> a = new Studi<>();  
        int i = a.myst(1, 3);  
    }  
}
```

Typ-Löschung (Type Erasure)

```
class Studi<T> {  
    T myst(T m, T n) { return n; }  
  
    public static void main(String[] args) {  
        Studi<Integer> a = new Studi<>();  
        int i = a.myst(1, 3);  
    }  
}
```

```
class Studi {  
    Object myst(Object m, Object n) { return n; }  
  
    public static void main(String[] args) {  
        Studi a = new Studi();  
        int i = (Integer) a.myst(1, 3);  
    }  
}
```

Folgen der Typ-Löschung: *new*

`new` mit parametrisierten Klassen ist nicht erlaubt!

```
class Fach<T> {  
    public T foo() {  
        return new T(); // nicht erlaubt!!!  
    }  
}
```

Grund: Zur Laufzeit keine Klasseninformationen über `T` mehr

Folgen der Typ-Löschung: *static*

`static` mit generischen Typen ist nicht erlaubt!

```
class Fach<T> {  
    static T t;           // nicht erlaubt!!!  
    static Fach<T> c;     // nicht erlaubt!!!  
    static void foo(T t) { ... }; // nicht erlaubt!!!  
}  
  
Fach<String> a;  
Fach<Integer> b;
```

Grund: Compiler generiert nur eine Klasse! Beide Objekte würden sich die statischen Attribute teilen (Typ zur Laufzeit unklar!).

Hinweis: Generische (statische) Methoden sind erlaubt. (Warum?)

Abgrenzung: Arrays vs. Generics: Reifizierung

Important

Arrays sind **reifiziert** (engl. *reified*): Sie “kennen” ihren Elementtyp **zur Laufzeit**.

```
String[] sa = new String[10];  
Object[] oa = sa;           // erlaubt: Array-Kovarianz  
  
oa[0] = "Hallo";           // ok  
oa[0] = new Double(2.0);   // Laufzeitfehler
```

- Arrays besitzen Typinformationen über gespeicherte Elemente
- Prüfung auf Typ-Kompatibilität zur **Laufzeit** (nicht Kompilierzeit!)
- Arrays sind **kovariant**: `String[] <: Object[]` wg. `String <: Object`

Generics + Arrays: Warum passt das nicht gut?

Important

=> Keine Arrays mit parametrisierten Klassen!

```
class Box<T> {  
    // T[] arr = new T[10];           // Compiler-Fehler  
}  
  
Foo<String>[] x = new Foo<String>[2]; // Compilerfehler  
  
Foo<String[]> y = new Foo<String[]>(); // OK :-)
```


Generics gibt es nur im Compiler, Arrays gibt es auch in der JVM mit vollem Typwissen.

- **Invarianz:** Generische Typen wie `List<T>` sind **invariant**
 - `List<Cat>` ist **kein** Untertyp von `List<Animal>`
 - `List<Animal>` ist **kein** Untertyp von `List<Cat>`
- **Kovarianz** mit `? extends T` → Producer (lesen)
 - Mit `S <: T`: `List<S> <: List<? extends S> <: List<? extends T> <: List<?>`
- **Kontravarianz** mit `? super T` → Consumer (schreiben)
 - Mit `S <: T`: `List<T> <: List<? super T> <: List<? super S> <: List<?>`
- **PECS:** *Producer Extends, Consumer Super*
- **Type Erasure:**
 - Generics gelten nur zur Compile-Time; Typ-Parameter werden zur Laufzeit gelöscht
 - Folgen: keine generischen Arrays, eingeschränktes instanceof, Bounds als Laufzeit-Ersatz
- Arrays sind **reifiziert** und **kovariant** → Laufzeit-Checks + `ArrayStoreException`



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

